

Towards CF Conventions conformance checking *on read* with cfdm

Sadie Bartholomew
Computational Scientist / Senior RSE
Covering work in collaboration with David Hassell

National Centre for Atmospheric Science (NCAS) Computational Modelling Services (NCAS-CMS)
and University of Reading, Meteorology Dept.
CF Conventions Workshop 2026 | 23rd September

Part 1 of 2

What is a CF Checker? What is available?



**National Centre for
Atmospheric Science**

NATURAL ENVIRONMENT RESEARCH COUNCIL

What is a CF Checker?

One definition is that it is a tool that tells us:

“Does this (any given) netCDF dataset follow the CF Conventions? If not, why not?”



CF Conformance Requirements and Recommendations 1.13 draft

- The following is a list of requirements and recommendations for a CF conforming netCDF file. They are organized by the section of the CF document that they pertain to.
- This document is intended to be a concise summary of the [CF Conventions document](#). If there are any discrepancies between the two, the conventions document is the ultimate authority.
- This document will be updated as required to correct mistakes or add new material required for completeness or clarity.

UGRID Conventions

Requirements and recommendations relating to the UGRID conventions for storing unstructured (or flexible mesh) data in netCDF files are not described here, but are nonetheless considered as part of the CF conformance requirements and recommendations. See <https://github.com/ugrid-conventions/ugrid-conventions> for the UGRID conformance requirements and recommendations.

2.1 Filename

Requirements:

- Filename must have ".nc" suffix.

2.2 Data Types

Requirements:

- Any text stored in a CF attribute or variable must be represented in Unicode Normalization Form C and encoded in UTF-8.
- Any attribute of variable-length string type must be a scalar (not an array).

2.3 Naming Conventions

Recommendations:

- Variable, dimension and attribute names should begin with a letter and be composed of letters (A-Z, a-z), digits (0-9),

“Each version of the CF conventions document is accompanied by a **conformance document** which lists the requirements and recommendations that could be verified by the compliance checker or any program with a similar aim.

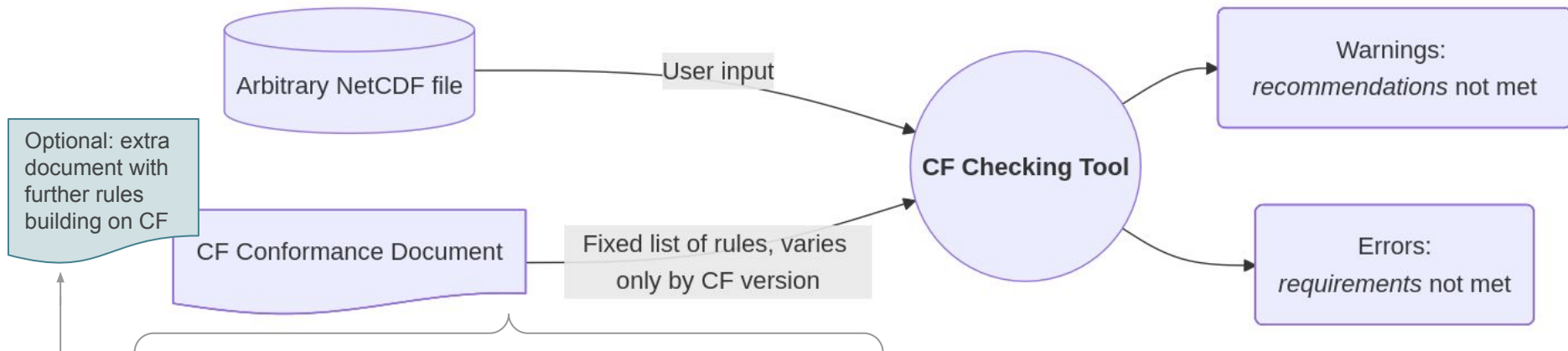
For any *recommendation* which is not followed, the checker gives a warning; for any *requirement* which is not met, it reports an error. ”

See document listing at:

<https://cfconventions.org/conventions.html>, shown is a screenshot of the latest (v1.13) document in HTML view (green shapes added by SB to highlight some components)



CF(+) Checking: automating validation against these criteria



- Nobody wants to read through the Conformance document and check against each criterion for every dataset (and that approach would be too prone to error)
- The *ideal* CF Checker is a general software tool (or part of one) that:
 - will produce these outputs (warnings and errors) from these inputs (any valid netCDF, at minimum, perhaps in future other file formats?)
 - **could be built upon to add further checking capability, as there is much interest for extension, e.g. CMOR or MIP-specific, etc. requirements**



CF(+) Checker tool as the *basis of metadata validation*: example for MIPs



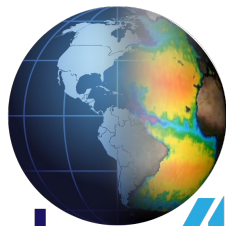
CMOR

'Climate Model Output Rewriter'
[no logo (?) so I used WordArt!]



The World Climate Research Programme's
Coupled Model Intercomparison Project

CMIP6, CMIP7, etc.

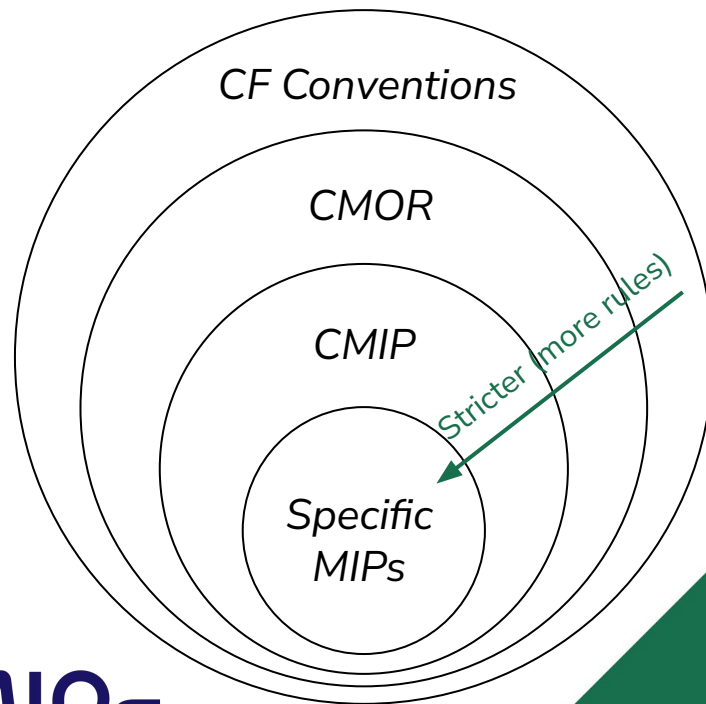


obs4MIPs

Observations for Model Intercomparisons Project

Specific MIPs, e.g. obs4MIPs

(Meta)data standardisation hierarchy:



What CF Checking tools are available and what is their status?

- **CEDA/NCAS CF Checker** (see github.com/cedadev/cf-checker), created by Ros Hatcher (of NCAS-CMS) and supported by CEDA
 - Multiple interfaces: CLI/command + browser UI hosted on a CEDA server
 - *Only maintained for up to CF v1.8* (released early 2020), with no development plans for versions beyond this (*stagnant* in that sense)
- **IOOS Compliance Checker** (see github.com/ioos/compliance-checker):
 - To be covered next in Filipe's talk!
- (WIP towards) **read-in compliance checking in the Python library cfdm:**
 - I will talk about this now
- Others?



Uploaded File: 12km_orog.nc

[Check another file](#)

CHECKING NetCDF FILE: /tmp/tmpkr71u4pt.nc

=====

Using CF Checker Version 4.1.0

Checking against CF Version CF-1.8

Using Standard Name Table Version 92 (2025-07-24T14:20:46Z)

Using Area Type Table Version 13 (20 March 2025)

Using Standardized Region Name Table Version 5 (12 November 2024)

WARN: (2.6.1): No 'Conventions' attribute present

Checking variable: rlon

Checking variable: rlat

Checking variable: surface

WARN: (3.1): units level is deprecated

Checking variable: t

ERROR: (3.1): Invalid units: days since 0000-01-01 00:00:00

WARN: (4.4.1): Use of the calendar and/or month_lengths attributes is recommended for time coordinate variables

Checking variable: ht

INFO: attribute title is being used in a non-standard way

Checking variable: rotated_pole

ERRORS detected: 1

WARNINGS given: 3

INFORMATION messages: 1

CEDA CF Checker: example response

Some notable content (enlarging so it is more readable):

Checking variable: t

ERROR: (3.1): Invalid units:
days since 0000-01-01 00:00:00

WARN: (4.4.1): Use of the
calendar and/or month_lengths
attributes is recommended for
time coordinate variables

...

ERRORS detected: 1

WARNINGS given: 3

INFORMATION messages: 1



Part 2 of 2

Towards CF checking on read with cfdm



**National Centre for
Atmospheric Science**

NATURAL ENVIRONMENT RESEARCH COUNCIL

The scaffolding for checking is already in place...

Screenshot of the cfdm docs

cfdm 1.13.3.0

A Python reference implementation of the CF data model

 31

Quick search

- Introduction
- CF data model
- Installation
- Contributing
- Tutorial
- API reference
- Philosophy
- Performance
- Extensions
- Releases
- Change log

cfdm development has been supported by the ERC through Couplet; by the EC Horizon 2020 programme through IS-ENES3; by NERC through UKFAFMIP; and by NCAS.



cfdm package

Introduction

Version 1.13.3.0 for version 1.13 of the CF conventions.

- [Functionality](#)
- [Performance](#)
- [Command line utilities](#)
- [Related packages](#)
- [Citation](#)
- [References](#)

The cfdm library implements the data model of the CF (Climate and Forecast) metadata conventions (<http://cfconventions.org>) and so should be able to represent and manipulate all existing and conceivable CF-compliant datasets.

The CF conventions are designed to promote the creation, processing, and sharing of climate and forecasting data using Network Common Data Form (netCDF) files and libraries (<https://www.unidata.ucar.edu/software/netcdf>). They cater for data from model simulations as well as from observations, made in situ or by remote sensing platforms, of the planetary surface, ocean, and atmosphere. For a netCDF data variable, they provide a description of the physical meaning of data and of its spatial, temporal, and other dimensional properties. The CF data model is an abstract interpretation of the CF conventions that is independent of the netCDF encoding.

For more details see *cfdm: A Python reference implementation of the CF data model* in the Journal of Open Source Software: <https://doi.org/10.21105/joss.02717>

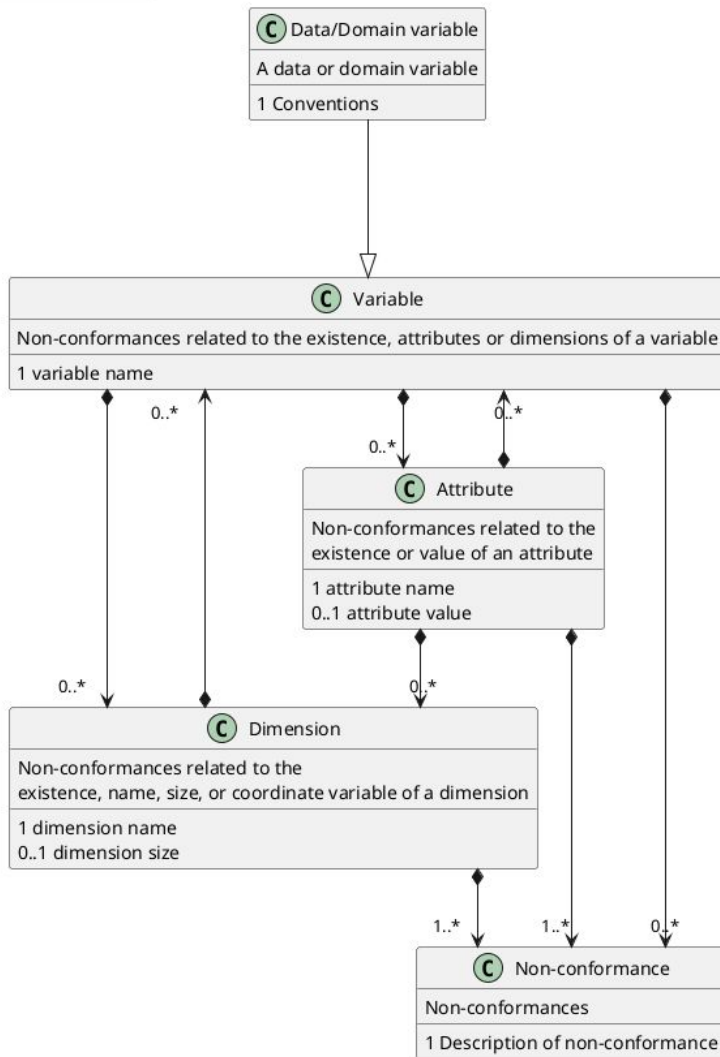
Functionality

The cfdm library can create field constructs *ab initio*, or read them from netCDF files, inspect, subspace and modify in memory, and write them to CF-netCDF dataset files. As long as it can interpret the data, cfdm does not enforce CF-compliance, allowing non-compliant datasets to be read, processed, corrected and rewritten.

It does not contain higher-level analysis functions (such as regridding) because the expectation is that

- Our Python data model library `cfdm` (see <https://github.com/NCAS-CMS/cfdm>) implements the CF Data Model and is guaranteed to be up-to-date for any CF release. It already has logic to process netCDF and many others formats taking their CF Compliance (or lack of!) into consideration
- We realised a ‘least efforts’ path to compliance checking would be adding compliance reporting logic into `cfdm`
- A major benefit is that users don’t have to check CF compliance/conformance separately, they can be **alerted to issues as and when they read/open their data**
- Compliance on `cfdm.read` call and can also be queried via the `dataset_compliance()` method on a field





A data model approach

- Want to note issues i.e. non-compliance by requirement or recommendation in a **structured way to allow hierarchical storing of problems which indicates the direct component and any children components which are affected** → so both the root of the issue, and any components indirectly involved, are clear
- To do this, we considered possible links between components: the 3 netCDF (Classic i.e. core) Data Model objects can map onto each other like so:
 - variable → attribute, dimension
 - attribute → variable, dimension
 - dimension → variable*
- Issues are ultimately stored as a string sentence 'reason' plus a code corresponding to the Conformance document numbering



field.dataset_compliance() output structures: example 1

- If a field is fully compliant*, the output is very simple: the only dictionary key is 'CF version' and reports the version of the CF Conventions which the field was checked against:

```
{ 'CF version': '1.13' }
```

- *at least, if no issues are detected - until we have full coverage over the CF Conformance document, this isn't the same thing! We emphasise 'preliminary' work:

Version 1.13.2.0

2026-06-29

- New methods to convert to xarray: `cfm.Field.to_xarray` and `cfm.Domain.to_xarray` ([#394](#))
- New output format for `cfm.write` that creates an xarray dataset in memory: `'XARRAY'` ([#394](#))
- Improved and restructured output of `cfm.Field.dataset_compliance` for (at present, preliminary) CF compliance checking, also available through a new keyword parameter to `cfm.read`: `_noncompliance_report` ([#365](#), [#366](#))
- Fix bug in `cfm.read` that prevented some OPeNDAP URLs being read ([#406](#))
- Fix bug in `cfm.write` when writing numpy scalars as Zarr attributes ([#408](#))
- New optional dependency: `xarray>=2026.2.0`



field.dataset_compliance() output structures: example 2

- If a field is not compliant in some way, output reports the issue(s) alongside this 'CF version' key. They are reported in a hierarchical structure abiding by the aforementioned data model.
- Simple example with one issue: the top-level field variable having an invalid standard name:

```
{ '/tas': { 'attributes': { 'standard_name': { 'non-conformance': [{ 'code': 400999,
                                                                    'reason': 'standard_name '
                                                                    'attribute '
                                                                    'has '
                                                                    'invalid '
                                                                    'name' }],
                                                                    'value': 'temp_of_air' } } },
  'CF_version': '1.13' }
```



field.dataset_compliance() output structures: example 3

- Given the hierarchical structure abiding by the data model, output dict can be pretty nested, but this is intended mostly for *'machine'* and the intention is to add logic to parse this to convert to tabular and graphical form (see next!) for *human* eyes
- Example shows a field from a UGRID file where invalid standard names have been set deliberately:

```
{'/v': {'attributes': {'mesh': {'value': '/mesh',  
                                'variables': {'/mesh': {'attributes': {'edge_node_connectivity': {'value': '/Mesh2_edge_nodes',  
                                                                                               'variables': {'/Mesh2_edge_nodes': {'attributes': {'standard_name': {'non-conformance': [{'code': 400999,  
                                                                                               'reason': 'standard_name '  
                                                                                               'attribute '  
                                                                                               'has '  
                                                                                               'invalid '  
                                                                                               'name'}]},  
                                                                                               'value': 'badname_dummy'}}}}}},  
                                'face_face_connectivity': {'value': '/Mesh2_face_links',  
                                                            'variables': {'/Mesh2_face_links': {'attributes': {'standard_name': {'non-conformance': [{'code': 400999,  
                                                                                               'reason': 'standard_name '  
                                                                                               'attribute '  
                                                                                               'has '  
                                                                                               'invalid '  
                                                                                               'name'}]},  
                                                                                               'value': 'badname_dummy'}}}}}},  
                                'face_node_connectivity': {'value': '/Mesh2_face_nodes',  
                                                            'variables': {'/Mesh2_face_nodes': {'attributes': {'standard_name': {'non-conformance': [{'code': 400999,  
                                                                                               'reason': 'standard_name '  
                                                                                               'attribute '  
                                                                                               'has '  
                                                                                               'invalid '  
                                                                                               'name'}]},  
                                                                                               'value': 'badname_dummy'}}}}}},  
                                'standard_name': {'non-conformance': [{'code': 400999,  
                                                                    'reason': 'standard_name '  
                                                                    'attribute '  
                                                                    'has '  
                                                                    'invalid '  
                                                                    'name'}]},  
                                                                    'value': 'badname_dummy'}}}}}},  
                                'standard_name': {'non-conformance': [{'code': 400999,  
                                                                    'reason': 'standard_name '  
                                                                    'attribute '  
                                                                    'has '  
                                                                    'invalid '  
                                                                    'name'}]},  
                                                                    'value': 'badname_v'}}}},  
                                'value': 'badname_v'}}}},  
                                'CF version': '1.13'}}
```



field.dataset_compliance() output structures: example 4

```
{'/ta': {'attributes': {'ancillary_variables': {'value': '/air_temperature_standard_error',
                                              'variables': {'/air_temperature_standard_error': {'attributes': {'standard_name': {'non-conformance': [{'code': 400999,
                                                                    'reason': 'standard_name '
                                                                    'attribute '
                                                                    'has '
                                                                    'invalid '
                                                                    'name'}]},
                                                                    'value': 'badname_air_temperature_standard_error'}}}}}},

'cell_measures': {'value': '/cell_measure',
                  'variables': {'/cell_measure': {'attributes': {'standard_name': {'non-conformance': [{'code': 400999,
                                                                    'reason': 'standard_name '
                                                                    'attribute '
                                                                    'has '
                                                                    'invalid '
                                                                    'name'}]},
                                                                    'value': 'badname_cell_measure'}}}}}},

'cell_methods': {'non-conformance': [{'code': 400999,
                                       'reason': 'standard_name '
                                       'attribute '
                                       'has '
                                       'invalid '
                                       'name'}]},
                  'value': 'badname_cellmethod'},

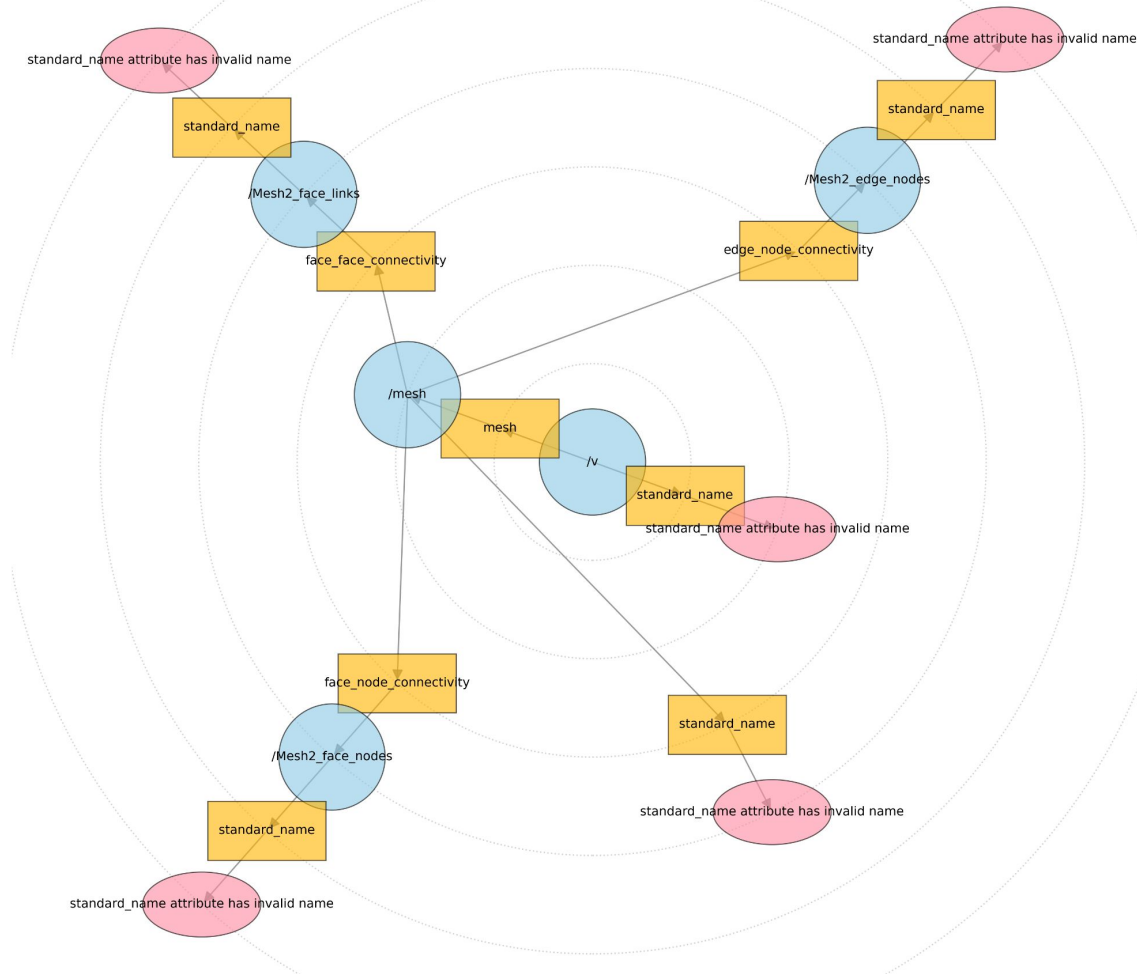
'coordinates': {'value': '/time',
                'variables': {'/auxiliary': {'attributes': {'standard_name': {'non-conformance': [{'code': 400999,
                                                                    'reason': 'standard_name '
                                                                    'attribute '
                                                                    'has '
                                                                    'invalid '
                                                                    'name'}]},
                                                                    'value': 'badname_auxiliary'}}}},
                  '/latitude_1': {'attributes': {'standard_name': {'non-conformance': [{'code': 400999,
                                                                    'reason': 'standard_name '
                                                                    'attribute '
                                                                    'has '
                                                                    'invalid '
                                                                    'name'}]},
                                                                    'value': 'badname_latitude_1'}}}},
                  '/longitude_1': {'attributes': {'standard_name': {'non-conformance': [{'code': 400999,
                                                                    'reason': 'standard_name '
                                                                    'attribute '
                                                                    'has '
                                                                    'invalid '
                                                                    'name'}]},
                                                                    'value': 'badname_longitude_1'}}}},
                  '/time': {'attributes': {'standard_name': {'non-conformance': [{'code': 400999,
                                                                    'reason': 'standard_name '
                                                                    'attribute '
                                                                    'has '
                                                                    'invalid '
                                                                    'name'}]},
                                                                    'value': 'badname_time'}}}}}},

'standard_name': {'non-conformance': [{'code': 400999,
                                       'reason': 'standard_name '
                                       'attribute '
                                       'has '
                                       'invalid '
                                       'name'}]},
                  'value': 'badname_ta'}}},

'CF version': '1.13'}
```



Machine output to human-friendly user output



We haven't developed proper code to convert the nested dict output to a human-friendly report, but we envisage code to generate nice tables and graphs (nodes and edges connecting chains of influence) from this. For example, left corresponds to example 3.



Machine output to human-friendly user output



We haven't developed proper code to convert the nested dict output to a human-friendly report, but we envisage code to generate nice tables and graphs (nodes and edges connecting chains of influence) from this. For example, left corresponds to example 4.



Status and future plans for read-in cfdm checking

- **Status:**

- compliance checking on read in cfdm is available as of v1.13.2.0 (i.e. now, though always try to use the latest version, at time of talk this is 1.13.3.0)...
- ...but it only checks against a very limited part of the Conformance document, such as validity of standard names against the current table

- **Future:**

- depends on funding and assigned projects, so at present it is unclear how fast compliance checking in cfdm can advance!



Summary

- A CF Checker answers “*Does this netCDF dataset follow CF at [version]? If not, why not?*” and acts as the basis of metadata validation not only for CF but standards building on it
- Checker tools validate against the Conformance Document which provides two levels: requirements (error if not met) and recommendations (warn if not met)
- There was a CEDA supported tool but that is stagnant and CF version ≤ 1.8 only, now the IOOS Compliance Checker is the comprehensive choice for a standalone tool. Both are available as a Python library or a CLI (command-line) format
- **Work is underway to incorporate a CF Checker to our Python data model library `cfdm` (see <https://github.com/NCAS-CMS/cfdm>) which already implements the data model and processes datasets according to that and the netCDF data models, so a lot of the hard work has already been done towards checking. As of `cfdm v1.13.2.0` (mid 2026) we have limited reporting and we are incrementally including more coverage against the conformance document going forward.**
- Compliance reported (optionally) as open a dataset. An advantage of this approach is validation at read time, not separately! Can also be queried via any field's `dataset_compliance()` method.

